Nexter

Архитектура, API, данные и эксплуатация

Личный финансовый учёт и криптоторговля

Версия документа: 2026-11-12

Включая проектирование торговых ботов (paper)

Сгенерировано из docs/project

Содержание

1. Обзор проекта
2. Архитектура
3. Модель данных
4. API
5. Эксплуатация
6. Безопасность
7. Troubleshooting
8. Гайд для AI-агентов
9. Торговые боты (design)

Обзор проекта

> Версия документа: 2026-11-12. Это живая документация: при изменении схемы, API или поведения обновляйте соответствующий раздел и `docs/decisions.md`.
>
> Быстрый старт для агента: `agent-guide.md`. Архитектура ботов: `trading-bots.md`. PDF-копия: `../pdf/nexter-project-guide.pdf`.

1. Что это за приложение

Nexter — личный финансовый учёт владельца. Один пользователь, без публичной регистрации. Приложение объединяет:

- криптопортфель и журнал сделок;
- DEX-свопы через WalletConnect/Tangem (BSC и Ethereum);
- каталог токенов (База токенов), watchlist-группы и котировки;
- статистику и отчёты по FIFO;
- личные финансы в BYN;
- задачи, заметки, привычки;
- end-to-end зашифрованное хранилище паролей;
- импорт исторических данных из Notion;
- модуль бумажной торговли ботами (`Тестирование`) с изолированным paper-ledger.

Приложение не является биржей, кастодиальным сервисом или налоговым калькулятором. Оно не хранит приватные ключи и seed-фразы. Все подписи выполняются в кошельке владельца.

2. Технологический стек

Слой	Технологии
Монорепозиторий	pnpm 10.20, TypeScript strict
API	NestJS 11, Fastify 5, Drizzle ORM, PostgreSQL 17/18, Zod, Pino, Sentry
Web	React 19, TanStack Router, TanStack Query, Tailwind CSS 4, Vite
Worker	Node.js + pg-boss
Тесты	Vitest (unit/integration), Supertest, Playwright (E2E)
Деньги	PostgreSQL `numeric`, `decimal.js`, строки в API
Очереди/фоновые задачи	pg-boss (worker), интервальные сервисы API (котировки, снапшоты, индексатор)
Деплой	Railway (API + worker + Postgres), Dockerfile
Бэкапы	`pg_dump` custom format + AES-256-GCM, restore drill

3. Структура репозитория

```
apps/
  api/        NestJS API; также раздаёт собранный SPA из apps/web/dist
  web/        React SPA
  worker/     pg-boss worker (импорт/фоновые задачи)
  e2e/        Playwright acceptance tests
packages/
  contracts/  Zod-схемы и TypeScript-типы API (единый контракт web/api)
  db/         Drizzle schema, миграции, скрипты backup/restore/provision
  ui/         общие UI-компоненты (если используются)
  config/     общий tsconfig/конфигурация
docs/
  project/    эта документация
  adr/        архитектурные решения
  runbooks/   операции, восстановление, деплой, импорт
  decisions.md история продуктовых/архитектурных решений
```

3.1. Поток запроса

```
Браузер (React SPA)
  → HTTPS /api/v1/*
  → Fastify + NestJS guards/interceptors
  → controller → service → Drizzle → PostgreSQL
  → JSON (Zod-контракт)
```

SPA и API живут на одном origin в production. Cookie-сессия (`HttpOnly`) + CSRF-токен (`nexter_csrf`) защищают мутации. Swagger доступен только при `ENABLE_DOCS=true` и `NODE_ENV=development/test` по `/api/docs`.

4. Основные подсистемы

Подсистема	Назначение	Ключевые таблицы
Auth	Вход, TOTP, сессии, CSRF, rate limit	`users`, `sessions`, `rate_limits`
Accounting	Счета, транзакции, проводки, FIFO-лоты, сделки	`accounts`, `transactions`, `postings`, `lots`, `lot_consumptions`, `trade_groups`, `trade_details`
Portfolio/Dashboard	Позиции, оценка, P&L, снапшоты, графики	`assets`, `market_quotes`, `portfolio_snapshots`
Market data	DexScreener/CoinGecko, кэш иконок, refresh	`market_quotes`, `assets`
Watchlist	База токенов, группы, контракты, метаданные	`watchlist`, `watchlist_groups`, `assets`
Swap	KyberSwap quote/build/simulate, WalletConnect, трекинг	`swap_orders`, `swap_order_close_groups`, `swap_events`, `token_approvals`
Imports	Notion legacy import, staging, preview/commit/rollback	`imports`, `import_rows`
Reports	Периоды, realized P&L, статистика, CSV	вычисляется из accounting
Finance	BYN-модуль: счета, категории, операции, переводы	`finance_accounts`, `finance_categories`, `finance_transactions`, `finance_transfers`
Tasks/Notes/Habits	Личные задачи, журнал, привычки	`tasks`, `task_images`, `notes`, `note_images`, `habits`
Vault	E2E-шифрование паролей в браузере	`vault_configs`, `vault_items`
Reconciliation	Read-only сверка с on-chain	`watched_addresses`, `indexed_transfers`, `reconciliation_runs/items`
Audit	Журнал всех мутаций	`audit_log`
Trading bots	Бумажная торговля стратегиями	`bot_strategies`, `trading_bots`, `bot_tokens`, `bot_positions`, `bot_trades`, `bot_equity_snapshots`, `bot_events`, `bot_quote_samples`

5. Ключевые принципы

1. **Один владелец.** Нет публичной регистрации. Все данные owner-scoped.
2. **Деньги — только строки/`numeric`.** В TS использовать `decimal.js`; не приводить деньги к `number`.
3. **Ledger append-only.** Исправления — новые транзакции, а не редактирование старых. Ручные операции и импорт пишут проводки и лоты.
4. **Каждая мутация аудируется.** `audit_log` пишется в той же транзакции, где меняются данные.
5. **Никаких приватных ключей/seed на сервере.** Подпись — только в кошельке владельца.
6. **Единый источник цен.** `market_quotes` (DexScreener 60с / CoinGecko 10мин) → `PricingService` → portfolio/dashboard/trades/watchlist. Не создавать отдельные источники цен без ADR.
7. **Русский UI, английский код.** Идентификаторы и комментарии — English; тексты интерфейса — Russian.
8. **Схема меняется только через ADR + миграцию.** Миграции аддитивные, с бэкапом и restore-проверкой.
9. **Секреты не попадают в репозиторий/логи.** Env-переменные, ключи, токены — только в Railway/локальном `.env`.
10. **Сначала preview, потом commit.** Импорт и опасные операции показывают preview/итоги; destructive-операции требуют подтверждения.

6. Быстрый локальный запуск

```
npx pnpm@10.20.0 install
npx pnpm@10.20.0 build:packages
npx pnpm@10.20.0 dev
```

Нужны `.env` (см. `apps/api/.env.example`, если есть) и PostgreSQL. Для production-подобной проверки:

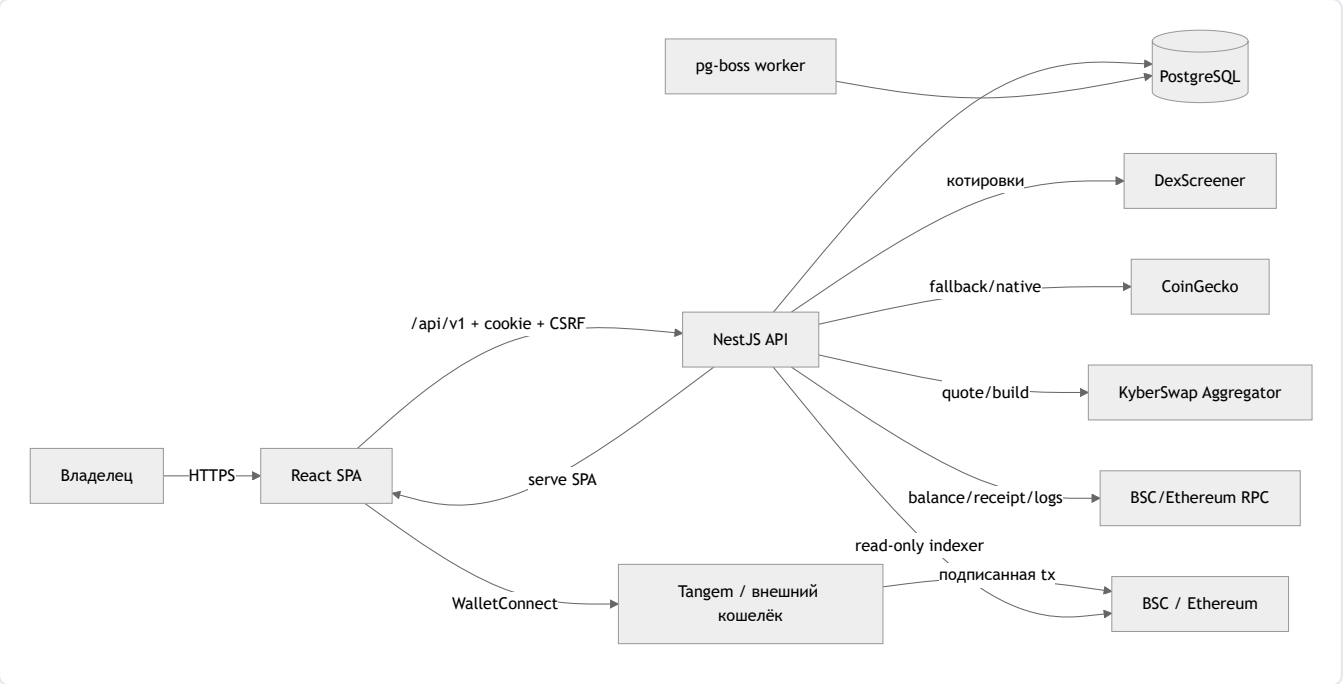
```
npx pnpm@10.20.0 format:check
npx pnpm@10.20.0 lint
npx pnpm@10.20.0 typecheck
npx pnpm@10.20.0 test
npx pnpm@10.20.0 test:e2e
npx pnpm@10.20.0 build
```

7. Где искать ответы

Вопрос	Документ
Как устроены модули и потоки данных	`architecture.md`
Какие таблицы и связи	`data-model.md`
Какие есть API-эндпоинты	`api.md`
Как деплоить, бэкапить, восстанавливать	`operations.md`
Модель угроз и ограничения	`security.md`
Что делать при типовых ошибках	`troubleshooting.md`
Как агенту безопасно менять код	`agent-guide.md`
Архитектура торговых ботов	`trading-bots.md`

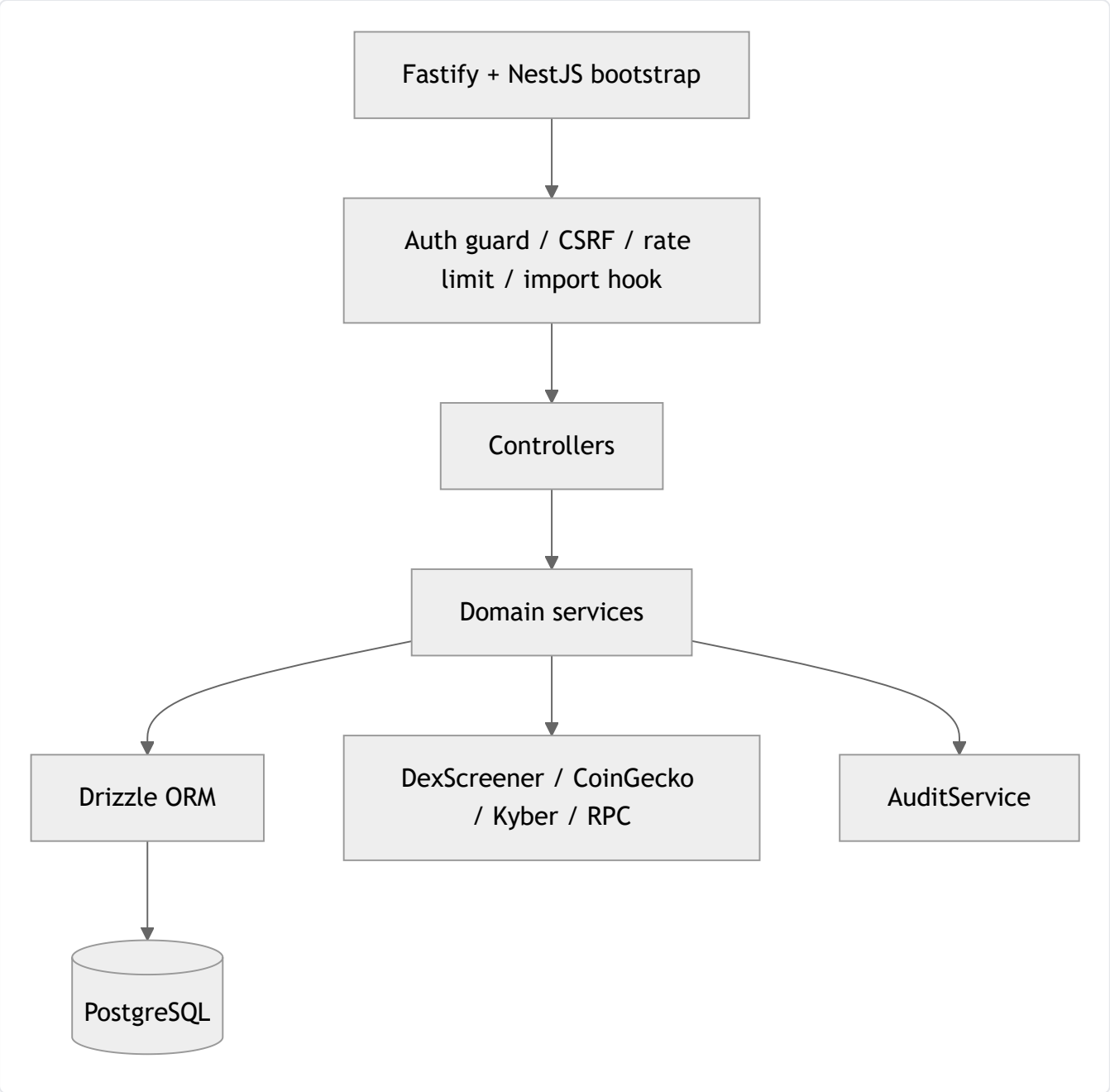
Архитектура

1. Контекст системы



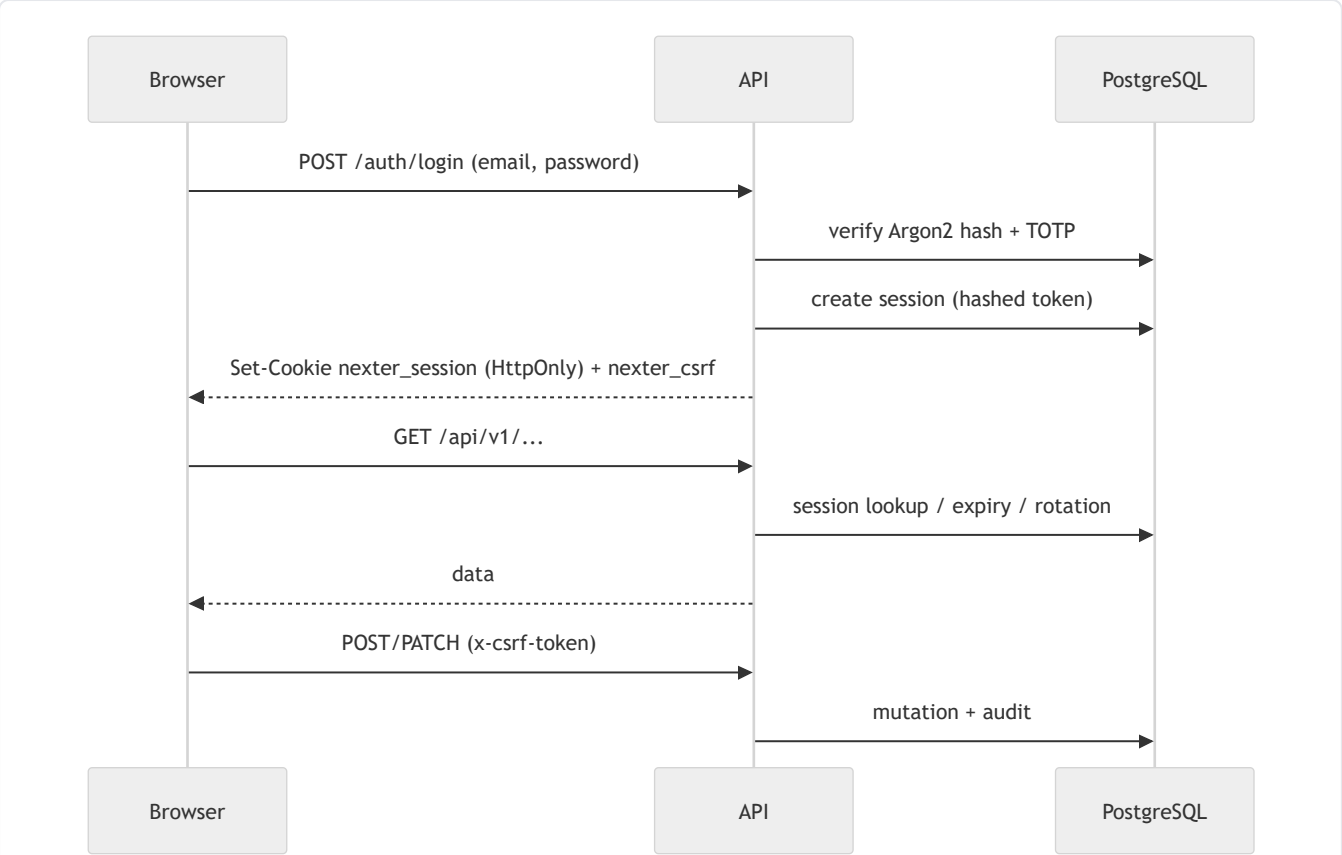
- **SPA** не хранит приватные ключи и не работает с БД напрямую.
- **API** — единственная точка записи в БД. Он же раздаёт статику SPA из `apps/web/dist`.
- **Worker** выполняет фоновые задачи pg-boss. Сейчас worker использует owner-роль БД, потому что pg-boss создаёт/меняет свою схему; API использует ограниченную роль `nexter_app`.
- **WalletConnect/Tangem** — единственный путь подписи. Сервер не может подписать транзакцию за владельца.
- **RPC/индексер** используется для проверки балансов, квитанций и reconciliation; он не является источником цен.

2. Слои API



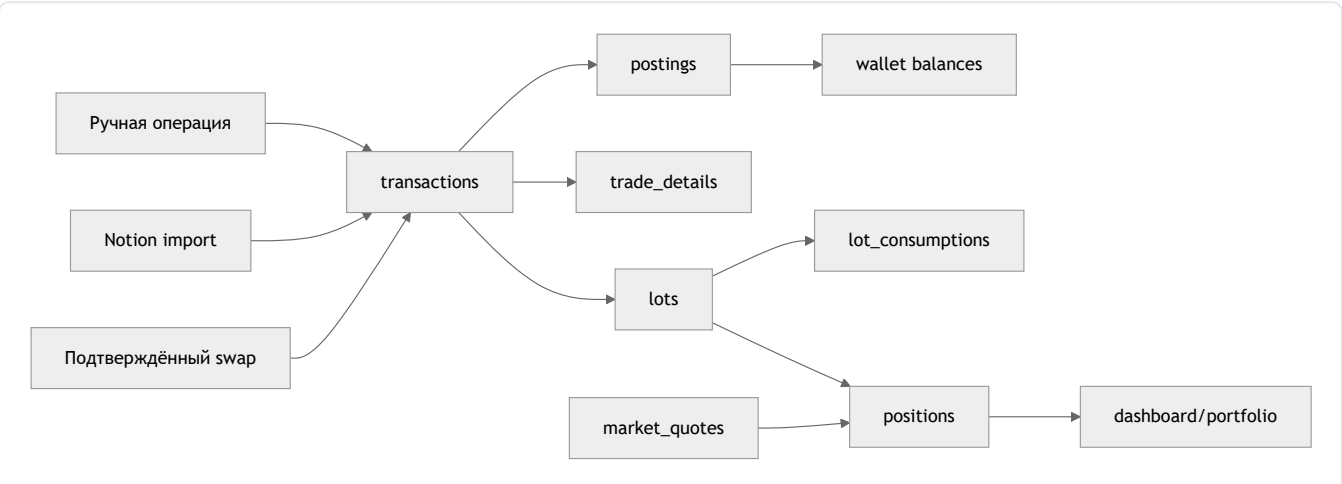
- **Controllers** только валидируют вход через Zod (`ZodValidationPipe`) и вызывают сервисы.
- **Services** содержат бизнес-логику, транзакции и аудит.
- **Drizzle** используется напрямую; отдельного repository-слоя нет.
- **AuditService** пишет в `audit_log`; для критичных мутаций используется `strict: true` (ошибка аудита откатывает мутацию).
- **Auth guard** проверяет `HttpOnly` cookie-сессию; мутации требуют `x-csrf-token`.
- **Rate limit** — in-memory, по IP; в тестах лимиты выше.

3. Аутентификация и сессия



- TOTP обязателен после настройки; step-up TOTP используется для свопов выше `SWAP_STEP_UP_THRESHOLD_USD`.
- Сессии хранятся хэшами; cookie `Secure`/`SameSite` настраиваются окружением.
- Восстановление пароля — через recovery-код; сброс инвалидирует сессии.
- Публичной регистрации нет; первичная настройка — по `SETUP_TOKEN`.

4. Учётный контур (ledger)



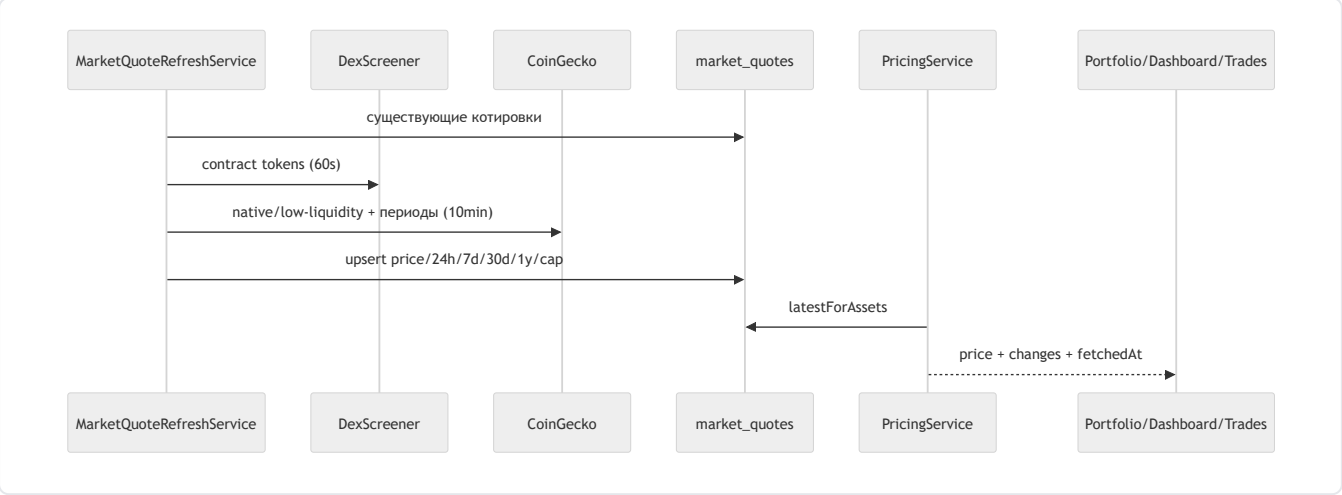
- Каждая операция создаёт `transactions` + `postings` (двойная запись по счетам).
- Покупка создаёт `lots`; продажа списывает FIFO и создаёт `lot_consumptions` с realized P&L.
- `trade_groups` объединяют лоты/сделки одного актива; `trade_details` хранит reference/execution цены.
- `accounts` бывают `wallet`, `external`, `income`, `expense`, `equity`.

- Кэш-эквиваленты (USDT/USDC) не создают лоты; они учитываются как cash.
- Ручные корректировки (`adjustment`) создают лот по текущей стоимости; отрицательных adjustment в UI нет. Для уменьшения используется withdrawal/продажа.
- Ledger append-only: ошибки исправляются новыми операциями. Технические удаления ошибочных строк допустимы только как аварийная операция с audit-записью и бэкапом.

4.1. FIFO

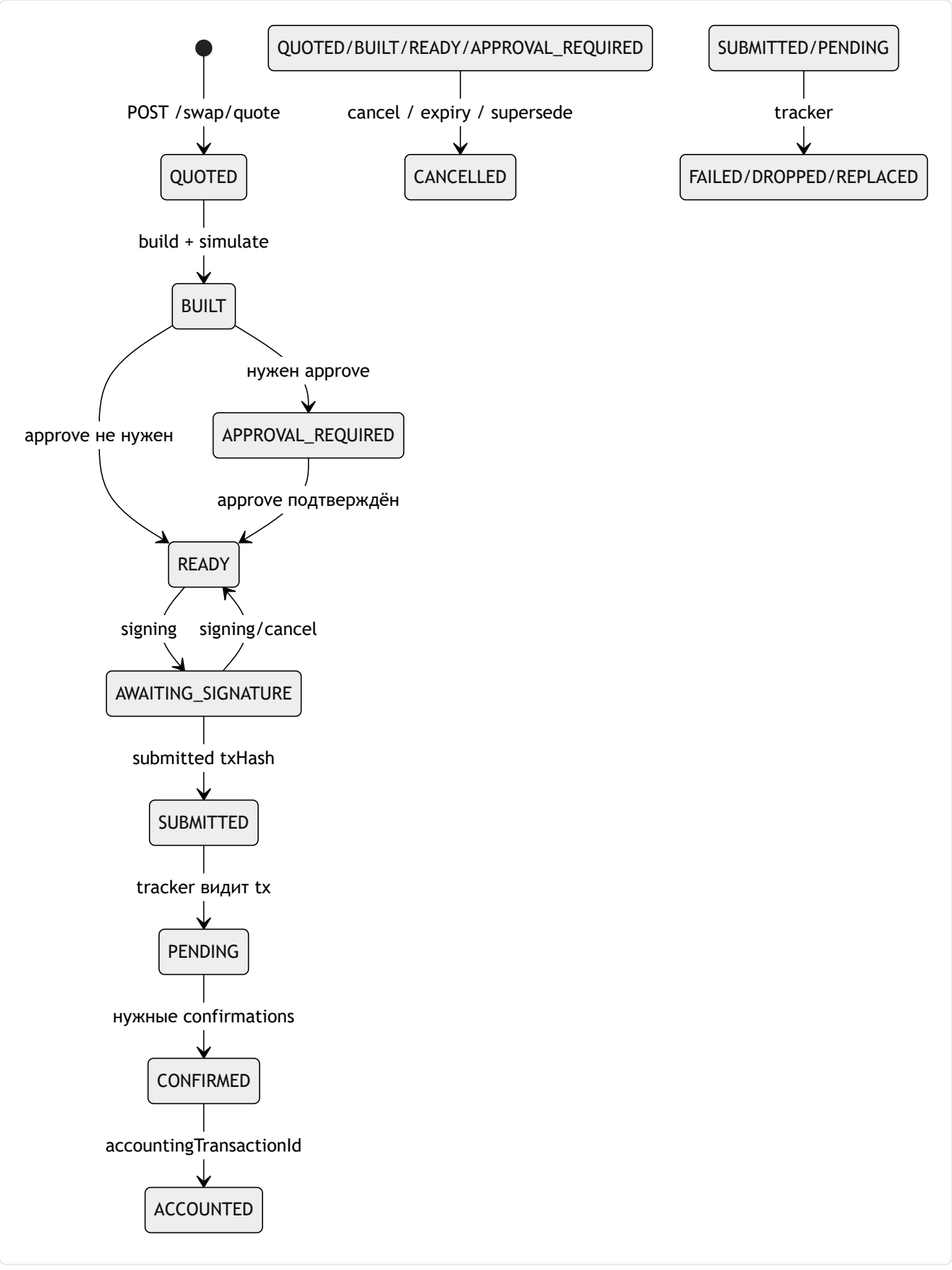
- Лоты сортируются по `acquiredAt, createdAt`.
- Продажа/списание списывает самые старые лоты.
- `remainingAmount`, `remainingCostBasisUsd` обновляются.
- Если лот закрыт не полностью — группа становится `partial`.
- Частичное закрытие сделки передаёт `closeGroupIds` и ожидаемое количество; сервер проверяет остаток.
- Газ/комиссии: капитализируются в cost basis при покупке (если `includeGasInCostBasis`), иначе учитываются как fee.

5. Рыночные данные и оценка



- **Приоритет:** DexScreener для контрактных токенов с ликвидностью ≥ `MARKET_QUOTE_MIN_LIQUIDITY_USD` (по умолчанию $25k); CoinGecko — fallback/native.
- `market_quotes.fetched_at` — время последней проверки; `changed_at` — время изменения цены.
- `PricingService.latestForAssets` сначала читает `market_quotes`, затем fallback на manual `prices`.
- 7d/30d/1y и market cap приходят из CoinGecko; у токенов без CoinGecko поля `null`.
- Иконки проксируются через `/api/v1/watchlist/icon` (allowlist CoinGecko hosts), кэш 24ч на сервере + 24ч в браузере; ошибки icon-route — `no-store`.
- 3-месячный период намеренно не показывается: CoinGecko не отдаёт ровно 90 дней.

6. Свопы (BSC/Ethereum)



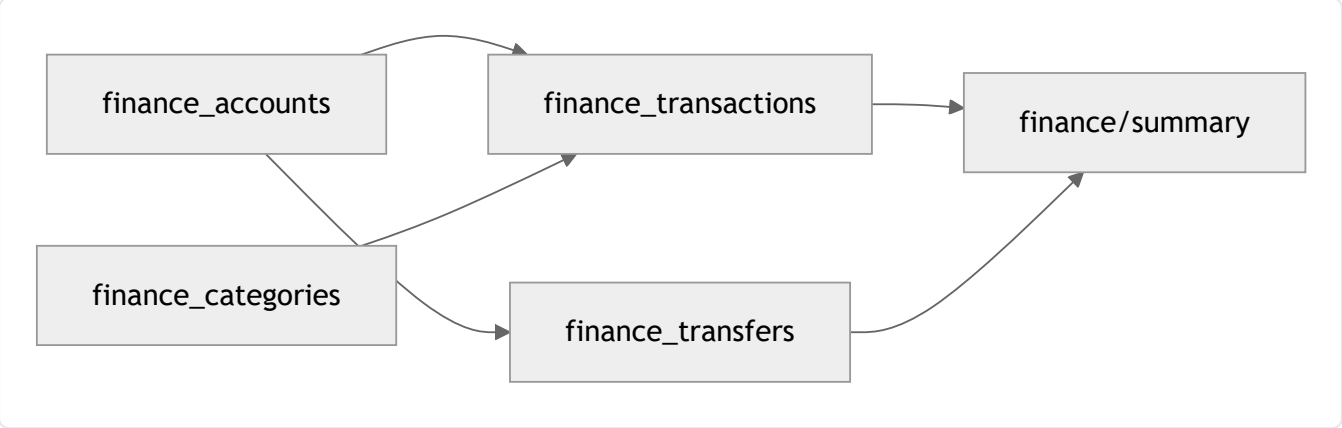
- Quote/build делает KyberSwap; сервер проверяет `amountIn`, router, calldata, selectors, recipient, minOut, executor, fees, deadline.

- `swap_order_close_groups` хранит выбранные группы и ожидаемое количество; сервер проверяет остаток и блокирует параллельные закрытия.
- Подпись — только WalletConnect/Tangem. Сервер не видит ключей.
- После `SUBMITTED` tracker опрашивает RPC, ждёт confirmations, проверяет receipt и создаёт accounting transactions.
- Если hash подписан, но submit не удался — `submit_failed`; повторная отправка использует тот же hash (без второй подписи).
- Если пользователь отклонил запрос, но кошелёк не вернул ошибку, UI позволяет нажать «Отменить ожидание»; подпись возвращается в READY, approval оставляется до ответа/expiry. Если hash всё же придёт, он будет отправлен на сервер.

7. Импорт Notion

- Legacy import: загрузка `source.json`, `orders.json`, `transactions.json` (raw Notion rows).
- `preview` парсит и проверяет, `detectExisting` сравнивает существующие лоты/группы, `commit` пишет ledger, `rollback` возможен только для последнего импорта без более поздних операций.
- Исторический 429-order control применяется только к исходному экспорту с numeric ID ≤ 466; более поздние экспорты используют preview totals и per-row detection (ADR 0032).
- `import_rows` хранит staging-строки; audit получает только сводку.
- Notion MCP — внешний инструмент для выгрузки; в репозитории нет токена Notion.

8. Финансы (BYN)



- Отдельный модуль, не влияет на крипто-ledger.
- Базовая валюта BYN; USD/EUR счета хранят native balance и курс.
- `credit` — liability; net worth = assets − debt.
- Операции отменяются (soft cancel), не удаляются; отменённые исключены из статистики.
- Импорт — preview/commit/rollback, staging в `finance_import_rows`.

9. Задачи, заметки, привычки

- `tasks`: soft archive, recurrence series (`series_id`, `recurrence`, `occurrence_index`), tags, images.
- Recurring update/delete применяется ко всей серии (content/delete); completion — per occurrence.
- `notes`: Markdown subset, soft archive, pin, images/attachments (data URL, 5 × 1.5 MB), notebook journal UI.
- `habits`: name + start date + archive; streak вычисляется в UI/API.
- Все мутации аудируются; тела заметок и ciphertext vault не логируются.

10. Vault

- Браузер: PBKDF2-SHA-256 (600k) → master key; случайный AES-256-GCM data key; элементы шифруются data key.

- Сервер хранит salt, iterations, wrapped key, ciphertext, IV; не может расшифровать.
- Мастер-пароль не сохраняется; потеря = потеря vault.
- Vault блокируется при reload; есть manual lock.
- UI: поиск по расшифрованным названию/логину/URL, пароль show/copy, архив/редактирование.

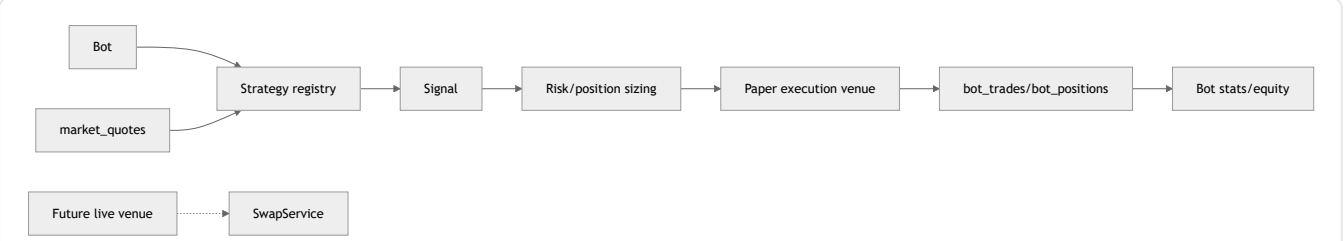
11. Reconciliation

- Read-only индексер on-chain transfers; `watched_addresses`, `indexed_transfers`, `reconciliation_runs/items`.
- Сверяет балансы/ledger с chain; не исправляет ledger автоматически.
- `RECONCILIATION_ENABLED` по умолчанию false.
- Data acceptance отложен до clean cutover (ADR 0014).

12. Аудит и наблюдаемость

- `audit_log`: action, entity, before/after, ip, userAgent, timestamp.
- Settings → Audit (`/settings/audit`) — просмотр и фильтр.
- `external_api_calls`: провайдер, endpoint, status, latency, errorCode.
- Pino JSON logs: request id, method, url без query, status, responseTime; секреты редактируются.
- Sentry — только safe error context.
- `/health` — liveness; `/ready` — DB readiness.

13. Бумажные торговые боты (см. `trading-bots.md`)



- `BotRunnerService` просыпается каждые 60 секунд, берёт advisory lock `nexter:bot-runner` на отдельном соединении и обрабатывает всех `running` ботов.
- Стратегии — код-реестр; параметры валидируются Zod. Paper execution считает комиссию/проскальзывание и пишет только в `bot_*` таблицы.
- `bot_quote_samples` даёт минутную историю для индикаторов; retention 30 дней.
- UI `/testing` показывает full-width карточки, статистику, позиции, сделки и события.
- Ключевое ограничение: реальная автоторговля не может использовать WalletConnect/Tangem без ручного подтверждения. Для live нужен отдельный подписант/session key/AA и отдельный security review; сервер не должен хранить приватные ключи.

Модель данных

Все таблицы создаются Drizzle-схемой `packages/db/src/schema/index.ts`; миграции лежат в `packages/db/drizzle/`. Деньги/количества — `numeric` (строки в API).

1. Пользователи и безопасность

Таблица	Назначение	Ключевые поля/связи
`users`	Владелец	id, email, passwordHash, totpSecret, recovery, timezone, baseCurrency
`sessions`	Сессии	userId, tokenHash, expiresAt, createdAt, lastUsedAt
`audit_log`	Аудит мутаций	userId, action, entityType, entityId, before/after JSONB, ip, userAgent
`rate_limits`	In-memory rate limit (таблица для персистентности)	key, points, windowStartedAt, blockedUntil
`settings`	Настройки владельца	userId, baseCurrency, displayCurrency, taxMethod, includeGasInCostBasis, timezone

`users` — один владелец. Все бизнес-таблицы содержат `user_id` и owner-scoped запросы.

2. Активы, сети, счета

Таблица	Назначение	Связи
`chains`	BSC, Ethereum (и TRON/Solana для manual holdings)	code unique
`assets`	Токены/монеты	chainId → chains, symbol unique per chain, contractAddress, coingeckoId, logoUrl, isCashEquivalent, isVerified
`accounts`	Счета	userId, type (`wallet` / `external` / `income` / `expense` / `equity`), chainId nullable
`watched_addresses`	Адреса для reconciliation	userId, chain, address
`indexed_transfers`	On-chain transfers	watched address, tx, amount, block
`reconciliation_runs/items`	Результаты сверки	run, item, status

Актив всегда привязан к сети. Один и тот же символ на BSC и Ethereum — два разных `assets` (например, USDC). `accounts` с типом `wallet` используются для позиций; системные счета (`external`, `income`, `expense`, `equity`) — для двойной записи.

3. Ledger

Таблица	Назначение	Связи
`transactions`	Заголовок операции	userId, type, subtype, source, importId, tradeGroupId, occurredAt, feesUsd, notes
`postings`	Проводки по счетам/активам	transactionId, accountId, assetId, amount, usdValue
`trade_groups`	Торговая группа/позиция	userId, assetId, status, openedAt, closedAt
`trade_details`	Детали сделки	tradeGroupId, transactionId, referencePriceUsd, executionCostUsd
`lots`	FIFO-лоты	userId, assetId, tradeGroupId, acquisitionTransactionId, amount, remainingAmount, costBasisUsd, remainingCostBasisUsd, acquiredAt
`lot_consumptions`	FIFO-списания	lotId, disposalTransactionId, amount, costBasisUsd, proceedsUsd, realizedPnlUsd
`prices`	Ручные/импортированные цены	assetId, priceUsd, observedAt, source
`market_quotes`	Последняя рыночная котировка	assetId PK, priceUsd, change24hPct, change7d/30d/1yPct, marketCapUsd, source, liquidityUsd, fetchedAt, changedAt
`portfolio_snapshots`	Дневной снимок капитала	userId, snapshotDate, cashUsd, positionsValueUsd, equityUsd

`transactions` + `postings` — append-only. `lots` / `lot_consumptions` — производные FIFO-структуры. Удаление старых транзакций запрещено; исправления — новые операции.

4. Импорт Notion

Таблица	Назначение
`imports`	Заголовок импорта (sourceHash, status, summary, committedAt)
`import_rows`	Staging-строки (source/order/transaction), status, data JSONB

`imports.sourceHash` защищает от повторного commit того же экспорта; `import_rows` удаляются при повторном preview/rollback.

5. Watchlist / База токенов

Таблица	Назначение
`watchlist_groups`	Группы (Основная, пользовательские)
`watchlist`	Карточка токена: assetId, groupId, thesis, tags, twitter/website, investors, priority, bscContractAddress, ethereumContractAddress, logoUrl override

`watchlist` — метаданные поверх `assets.bscContractAddress` / `ethereumContractAddress` — справочные адреса (ADR 0033), не меняют identity актива.

6. Swap

Таблица	Назначение
`swap_orders`	Ордер: chain, tokens, amounts, status, calldata, router, txHash, deadline, quote expiry, approval linkage, accountingTransactionId
`swap_order_close_groups`	Связь close-ордера с trade_groups + expected_amount
`swap_events`	События state machine (quote_created, built, ready, signing, signing_cancelled, submitted, confirmed, failed, ...)
`token_approvals`	Approve-транзакции: token/spender/amount/status/txHash

`swap_orders` — не ledger; accounting создаётся только после `CONFIRMED` + подтверждения receipt.

7. Финансы (BYN)

Таблица	Назначение
`finance_accounts`	Счета (cash/card/bank/credit), валюта, opening balance, credit limit
`finance_categories`	Категории income/expense + цвет
`finance_transactions`	Операции: kind, amount, currency, account, category, FX rate, cancelledAt
`finance_transfers`	Переводы между счетами (card_topup, credit, ...)
`finance_imports` / `finance_import_rows`	Preview/commit/rollback импорта

Модуль изолирован от крипто-ledger. Кредит — liability; net worth = assets − debt.

8. Задачи, заметки, привычки

Таблица	Назначение
`tasks`	title, description, status, priority, tags, dueAt, completedAt, relatedAssetId, seriesId, recurrence, recurrenceEndsAt, occurrenceIndex, archivedAt
`task_images`	attachments tasks (data URL, 5 × 1.5 MB)
`notes`	title, body Markdown, tags, pinned, relatedAssetId/TradeGroupId, archivedAt
`note_images`	attachments notes (images/files, data URL, 5 × 1.5 MB)
`habits`	name, startedOn, archivedAt

`tasks.seriesId` объединяет повторяющиеся occurrences; `occurrenceIndex` — номер. `note_images`/`task_images` хранят MIME и data URL; лимит проверяется сервисом.

9. Vault

Таблица	Назначение
`vault_configs`	userId unique, salt, iterations, wrappedDataKey, iv
`vault_items`	userId, ciphertext, iv, createdAt/updatedAt, archivedAt

Сервер не хранит мастер-пароль и не может расшифровать данные. Поиск выполняется в браузере после unlock.

10. Диагностика

Таблица	Назначение
`external_api_calls`	provider, endpoint, status, latencyMs, errorCode, createdAt

Используется для диагностики лимитов KyberSwap/CoinGecko; не содержит секретов.

11. Бумажные торговые боты

Таблица	Назначение
`bot_strategies`	Экземпляр стратегии: key registry, name, description, version, params jsonb, archivedAt
`trading_bots`	Бот: strategyId, status, budget/cash, maxPositions, positionSizePct, fee/slippage bps, daily loss, cooldown, peakEquity/maxDrawdown, timestamps
`bot_tokens`	Разрешённые токены бота; `(botId, assetId)` unique
`bot_positions`	Открытая paper-позиция: amount, avgCostUsd, costBasisUsd; average-cost учёт
`bot_trades`	Paper-сделки: side, amount, fill price, usdValue, fee, slippage, realizedPnl, reason, signal
`bot_equity_snapshots`	Снимки equity при сделках и раз в 5 минут
`bot_events`	Журнал created/started/stopped/buy/sell/error/reset/archived
`bot_quote_samples`	Минутные цены токенов активных ботов для индикаторов и истории

Таблицы изолированы от реального ledger: paper-сделки не пишут в `transactions`, `postings`, `lots` или `trade_groups`. `budgetUsd` неизменяем после создания; `reset` возвращает `cashUsd` к бюджету и удаляет paper-позиции/сделки/снимки. Полное описание — в `trading-bots.md`.

API

Базовый путь: `/api/v1`. Все ответы — JSON (кроме CSV/иконок). Контракты описаны Zod-схемами в `packages/contracts`; Swagger доступен локально по `/api/docs` при `ENABLE_DOCS=true` и `NODE_ENV=development/test`.

1. Общие правила

- Аутентификация: cookie `nexter_session` (HttpOnly, Secure в production).
- Мутации (`POST`, `PATCH`, `DELETE`) требуют header `x-csrf-token` со значением cookie `nexter_csrf`.
- Owner-only: все данные фильтруются по `userId`.
- Ошибки: `json { "error": { "code": "VALIDATION_ERROR", "message": "...", "details": {}, "requestId": "..." } }`
- Дефолтный `Cache-Control: no-store` для `/api/v1/*`, кроме `/api/v1/watchlist/icon` (public 24h).
- Деньги — строки; не использовать JS `number` для финансовых расчётов.
- Idempotency: ручные транзакции и импорт поддерживают `Idempotency-Key`/sourceHash.

2. Health

Метод	Путь	Описание
GET	`/health`	liveness
GET	`/ready`	readiness + DB

3. Auth

Метод	Путь	Описание
POST	`/auth/setup`	первичная настройка owner (setup token)
POST	`/auth/login`	email + password
POST	`/auth/2fa`	TOTP-проверка при входе
POST	`/auth/2fa/setup`	начать настройку TOTP
POST	`/auth/2fa/enable`	подтвердить TOTP
POST	`/auth/logout`	завершить сессию
GET	`/auth/me`	текущий owner/состояние 2FA
POST	`/auth/recovery`	восстановление по коду
POST	`/auth/reset-password`	сброс пароля
POST	`/auth/step-up`	step-up TOTP для свопов

4. Accounting

Метод	Путь	Описание
GET	`/accounts`	счета
GET	`/accounts/:id`	счёт
GET	`/assets`	все активы
GET	`/assets/:id`	актив
GET	`/positions`	позиции + cash/equity/missingPrices
GET	`/positions/:assetId`	позиция по активу
GET	`/portfolio/assets/:assetId`	полная карточка актива (position/quote/lots/trades/transactions/prices)
GET	`/dashboard/summary`	сводка
GET	`/dashboard/equity-curve`	equity curve
GET	`/dashboard/allocation`	аллокация
GET	`/dashboard/pnl`	месячный P&L
GET	`/dashboard/active-positions`	активные позиции
GET	`/dashboard/recent-trades`	последние сделки
GET	`/dashboard/data-quality`	quality issues
GET	`/transactions`	журнал операций (cursor/page)
POST	`/transactions`	ручная операция (buy/sell/swap/transfer/deposit/withdrawal/income/adjustment)
GET	`/trade-groups`	сделки/группы (filters: status, assetId, pnl, view, period)
GET	`/trade-groups/:id`	детали группы (lots, consumptions, audit, legs)
PATCH	`/trade-groups/:id/notes`	заметка группы
POST	`/trade-groups/:id/sell`	ручная продажа группы
GET	`/prices/:assetId`	история ручных цен
POST	`/prices/:assetId`	установить ручную цену fallback
GET	`/reports`	period report (`from`, `to`)
GET	`/export/:dataset.:format`	CSV export (transactions/trades/etc.)
POST	`/dashboard/snapshot`	ручной snapshot

5. Watchlist / База токенов

Метод	Путь	Описание
GET	`/watchlist`	список (cursor, groupId, includeArchived, search)
POST	`/watchlist`	добавить существующий asset
PATCH	`/watchlist/:id`	изменить карточку (thesis/tags/links/investors/contracts/logo/archive)
DELETE	`/watchlist/:id`	архив
GET	`/watchlist/groups`	группы
POST	`/watchlist/groups`	создать группу
PATCH	`/watchlist/groups/:id`	изменить группу
DELETE	`/watchlist/groups/:id`	удалить группу (перевод в Основную)
GET	`/watchlist/search?chain=&address=`	on-chain metadata + CoinGecko/DexScreener + alreadyInWatchlist
POST	`/watchlist/from-contract`	создать/привязать asset и добавить в watchlist
GET	`/watchlist/icon?url=`	прокси иконки (CoinGecko hosts, 24h cache)

6. Swap

Метод	Путь	Описание
GET	`/swap/config`	enabled/kill switch/chains/tokens/wallet allowlist
POST	`/swap/preview`	non-persisting quote (5s cache, 500ms debounce on client)
POST	`/swap/quote`	persist quote order (`closeGroupIds` для продажи)
POST	`/swap/build`	build + simulate calldata
POST	`/swap/approvals/build`	build approve
POST	`/swap/approvals/submitted`	attach approval tx hash
GET	`/swap/approvals`	approvals list
POST	`/swap/orders/:id/ready`	READY после approval
POST	`/swap/orders/:id/signing`	AWAITING_SIGNATURE
POST	`/swap/orders/:id/signing/cancel`	вернуть READY после отклонения
POST	`/swap/orders/:id/submitted`	attach swap tx hash
GET	`/swap/orders`	список ордеров
GET	`/swap/orders/:id`	детали ордера
POST	`/swap/orders/:id/cancel`	отменить unsigned order

7. Imports

Метод	Путь	Описание
GET	`/imports`	история импортов
GET	`/imports/:id`	импорт + rows
GET	`/imports/:id/reconciliation`	сверка
POST	`/imports/notion/preview`	preview (content-type `application/x-notion-import+json`)
POST	`/imports/:id/commit`	commit
POST	`/imports/:id/rollback`	rollback (только последний)

8. Finance (BYN)

Метод	Путь	Описание
GET	`/finance/accounts`	счета
POST	`/finance/accounts`	создать счёт
PATCH	`/finance/accounts/:id`	изменить/архивировать
GET	`/finance/categories`	категории
POST	`/finance/categories`	создать
PATCH	`/finance/categories/:id`	изменить
GET	`/finance/transactions`	журнал (period, cancelled)
POST	`/finance/transactions`	операция
PATCH	`/finance/transactions/:id/cancel`	отменить
PATCH	`/finance/transactions/:id/restore`	вернуть
GET	`/finance/transfers`	переводы
POST	`/finance/transfers`	перевод
GET	`/finance/summary`	сводка (`from/to` или `all=true`)
POST	`/finance/import/preview`	preview
POST	`/finance/import/commit`	commit
POST	`/finance/import/rollback`	rollback

9. Tasks / Notes / Habits

Метод	Путь	Описание
GET	`/tasks`	список (status/search/range/archived)
POST	`/tasks`	создать (recurrence → series)
PATCH	`/tasks/:id`	изменить (`applyToSeries` default true для series)
POST	`/tasks/:id/images`	attachment
DELETE	`/tasks/:id/images/:imageId`	удалить attachment
GET	`/notes`	список (search/archive/offset/limit)
GET	`/notes/:id`	детали + attachments
POST	`/notes`	создать
PATCH	`/notes/:id`	изменить/архивировать/pin
POST	`/notes/:id/images`	attachment (image/file)
DELETE	`/notes/:id/images/:imageId`	удалить attachment
GET	`/habits`	привычки
POST	`/habits`	создать
PATCH	`/habits/:id`	изменить/архивировать

10. Vault

Метод	Путь	Описание
GET	`/vault`	config + items (ciphertext)
POST	`/vault/initialize`	salt/iterations/wrapped key
POST	`/vault/items`	создать encrypted item
PATCH	`/vault/items/:id`	изменить ciphertext
DELETE	`/vault/items/:id`	архив

11. Audit / Settings / Reconciliation

Метод	Путь	Описание
GET	`/audit-log`	audit entries (action/entityType/period/cursor)
GET	`/settings`	настройки
PATCH	`/settings`	изменить
GET	`/reconciliation/runs`	запуски сверки
GET	`/reconciliation/runs/:id`	детали
POST	`/reconciliation/runs`	запустить (если enabled)
GET	`/reconciliation/addresses`	watched addresses

12. Trading bots (paper)

Метод	Путь	Описание
GET	`/trading-bots/strategies`	шаблоны registry + созданные стратегии
POST	`/trading-bots/strategies`	создать стратегию (`key`, `name`, `description`, `params`)
PATCH	`/trading-bots/strategies/:id`	изменить/архивировать
GET	`/trading-bots/bots`	список ботов + статистика (`includeArchived`)
POST	`/trading-bots/bots`	создать бота
GET	`/trading-bots/bots/:id`	бот + стратегия + токены + позиции
PATCH	`/trading-bots/bots/:id`	настройки (только не-running)
POST	`/trading-bots/bots/:id/start`	запуск
POST	`/trading-bots/bots/:id/stop`	остановка
POST	`/trading-bots/bots/:id/reset`	сброс paper-состояния
DELETE	`/trading-bots/bots/:id`	soft archive
GET	`/trading-bots/bots/:id/trades?limit=`	сделки
GET	`/trading-bots/bots/:id/events?limit=`	события
GET	`/trading-bots/bots/:id/equity?limit=`	equity curve

Бюджет — от $100 и кратен $100; токены должны быть активными записями Базы токенов. Running-боты не редактируются; стратегию с running-ботами нельзя архивировать. Подробности — в `trading-bots.md`.

Эксплуатация

1. Окружения и деплой

- Production: Railway project `generous-radiance`, API + worker + Postgres. API URL: `https://nexterapi-production.up.railway.app`.
- API собирается из `Dockerfile`; pre-deploy запускает `node packages/db/dist/migrate.js` с `MIGRATION_DATABASE_URL`.
- API раздаёт SPA из `apps/api/public`; Railway watch patterns API: `apps/api/**`, `apps/web/**`, `packages/**`.
- Worker собирается отдельно; watch patterns уже, worker не пересобирается на frontend-only изменения.
- Production API использует роль `nexter_app` (SELECT/INSERT/UPDATE, ограниченный DELETE). Worker использует owner-роль, потому что pg-boss делает DDL.
- `ENABLE_DOCS=true` включает `/api/docs` только в development/test.

2. Переменные окружения

Основные (см. `apps/api/src/shared/config/env.ts`):

Переменная	Назначение
`DATABASE_URL`	подключение API к БД
`MIGRATION_DATABASE_URL`	pre-deploy миграции (owner/DDL)
`SETUP_TOKEN`	первичная настройка owner
`AUTH_ENCRYPTION_KEY`	шифрование TOTP/recovery/сессий
`APP_URL`	origin для cookie/CSRF/CORS
`SESSION_TTL_DAYS`	срок сессии
`ENABLE_DOCS`	Swagger
`SWAP_ENABLED`	включить swap-модуль
`SWAP_EXECUTION_ENABLED`	разрешить execution (BSC)
`SWAP_ETHEREUM_EXECUTION_ENABLED`	разрешить execution Ethereum
`SWAP_KILL_SWITCH`	аварийный стоп
`SWAP_STEP_UP_THRESHOLD_USD`	порог step-up TOTP
`SWAP_WALLET_ALLOWLIST`	разрешённые wallet addresses
`SWAP_ROUTER_ALLOWLIST` / `SWAP_EXECUTOR_ALLOWLIST`	allowlist контрактов
`MARKET_QUOTES_ENABLED`	фоновые котировки
`MARKET_QUOTE_INTERVAL_SECONDS`	DEX refresh (60)
`MARKET_QUOTE_COINGECKO_INTERVAL_SECONDS`	CoinGecko refresh (600)
`COINGECKO_API_KEY`	optional demo/pro key
`BSC_RPC_URL` / `ETHEREUM_RPC_URL`	RPC
`WALLETCONNECT_PROJECT_ID`	WalletConnect
`RECONCILIATION_ENABLED`	on-chain indexer
`PORTFOLIO_SNAPSHOTS_ENABLED`	daily snapshots
`BOT_RUNNER_ENABLED`	paper-runner ботов (default true)
`BOT_RUNNER_INTERVAL_SECONDS`	интервал тика (default 60)
`BOT_SAMPLE_RETENTION_DAYS`	retention samples (default 30)
`BACKUP_ENCRYPTION_KEY`	только для backup/restore, не в API

Секреты хранятся в Railway Variables и локальном `.env`, не в репозитории.

3. Миграции

```
npx pnpm@10.20.0 db:generate   # после изменения schema
npx pnpm@10.20.0 db:migrate    # локально
```

- Миграции аддитивные; destructive изменения требуют ADR, бэкапа и явного approval.
- Перед production-миграцией: 1. сделать encrypted backup; 2. проверить restore на временной БД; 3. применить миграцию pre-deploy; 4. проверить `/ready`, схему, гранты `nexter_app`; 5. записать миграцию/бандл в `docs/decisions.md`.
- Production restore-проверка: `pg_restore` в `nexter_restore_check_*`, сравнить counts/sums, затем `dropdb`.

4. Бэкапы

```
DATABASE_URL=... BACKUP_ENCRYPTION_KEY=... BACKUP_OUTPUT_DIR=./backups \
  node packages/db/scripts/backup.mjs
```

- Формат: `NEXTER1 | IV | tag | ciphertext` (AES-256-GCM).
- Ключ вне репозитория (`~/.nexter/backup-key` локально; Railway secret в production).
- RPO 1 час, RTO 4 часа. Одноразовый дамп не заменяет расписание/PITR.
- Restore: stop writes → decrypt → `pg_restore` → verify counts/sums → switch traffic.
- Не хранить plaintext-дампы дольше проверки.

5. Мониторинг

- `/health` — процесс жив.
- `/ready` — БД доступна.
- Railway logs: pino JSON; искать `status=5xx`, `auth.rate_limited`, `SWAP_*`, `IMPORT_*`, `Market quote upsert failed`.
- Sentry: 5xx/неожиданные ошибки (секреты редактируются).
- `external_api_calls`: 429/5xx KyberSwap/CoinGecko.
- `swap_events`: state machine обменов.
- `audit_log`: бизнес-мутации.
- `/settings/data-quality`: negative balances, unverified assets, missing gas.

6. Feature flags и аварийные переключатели

- `SWAP_KILL_SWITCH=true` — остановить execution.
- `SWAP_EXECUTION_ENABLED=false` — запретить BSC execution.
- `SWAP_ETHEREUM_EXECUTION_ENABLED=false` — запретить Ethereum.
- `MARKET_QUOTES_ENABLED=false` — остановить refresher (котировки останутся последними).
- `RECONCILIATION_ENABLED=false` — остановить indexer.
- `PORTFOLIO_SNAPSHOTS_ENABLED=false` — остановить snapshots.
- `BOT_RUNNER_ENABLED=false` — остановить paper-ботов без остановки API. Позиции и история сохраняются; боты остаются `running`, но не тикают. Для полного стопа лучше остановить каждого бота в UI.
- `BOT_RUNNER_INTERVAL_SECONDS` — увеличить при нагрузке; стратегии увидят более редкие тики.
- `BOT_SAMPLE_RETENTION_DAYS` — уменьшить при росте `bot_quote_samples`.

7. Rollback

- Frontend-only: redeploy previous image; миграции обычно аддитивные, могут остаться.
- API/worker: redeploy previous image; проверить `/ready` и ключевые страницы.
- БД: предпочтителен forward fix; restore только для destructive-миграции/инцидента.
- При rollback свопа: сначала drain active/unaccounted orders, затем менять image; иначе старый image не сможет корректно трекать Ethereum orders.

8. Регулярные проверки

- ежедневно: `/ready`, 5xx, auth anomalies, backup exists/plausible, no negative balances/unverified spike, confirmed swaps with null accounting.
- еженедельно: audit anomalies, quote freshness, provider 429 rate.
- ежеквартально: restore drill; review secrets/roles.

9. Известные эксплуатационные ограничения

- Worker использует owner role (pg-boss DDL).
- PITR/ежедневный cron бэкап — residual; настроить до go-live.
- In-memory rate limit/quote guard не масштабируется на несколько API-реплик; при масштабировании нужен Redis/Postgres advisory lock.
- Reconciliation не исправляет ledger автоматически.
- Native TRON/Solana — manual-only, не входят в swap/indexer.
- Paper-боты работают только при живом API; worker их не запускает. Один тик защищён advisory lock, но при нескольких репликах все они должны иметь доступ к одной БД.
- `bot_quote_samples`, `bot_events` и `bot_equity_snapshots` очищаются по retention на каждом тике; при остановленном runner очистка не выполняется.
- Боты не влияют на реальный ledger; их P&L нельзя использовать для налогов или отчётов.

Безопасность

1. Модель угроз

Приложение — single-owner, self-hosted. Основные риски:

- компрометация owner-аккаунта/сессии;
- кража приватных ключей/seed (в приложении их нет, но владелец может хранить их в vault);
- подмена calldata/адресов при свопе;
- ошибки импорта/ledger, портящие финансовые данные;
- утечка секретов через логи/репозиторий;
- XSS/CSRF/SSRF через пользовательские URL/Markdown/иконки;
- provider rate limits/DoS.

2. Ключевые меры

Область	Меры
Auth	Argon2, TOTP, HttpOnly cookie, CSRF, rate limit, session rotation, recovery codes
Swap	allowlist кошельков/роутеров/executor/selectors, calldata decode, exact approvals, simulation, minOut, deadline, price impact cap, step-up TOTP, kill switch
Private keys	никогда не попадают на сервер; подпись только в кошельке
Money	`numeric`/Decimal, без float, проверка decimals/positive
Audit	каждая мутация, strict audit для критичных
Import	preview/commit/rollback, sourceHash, owner-scoped staging, no secrets in logs
Vault	E2E AES-256-GCM, PBKDF2 600k, мастер-пароль не сохраняется
Notes	Markdown subset без HTML, ссылки http/https, attachments allowlist + лимиты
Icons	SSRF allowlist CoinGecko hosts, timeout, content-type/size check
Logs	pino redaction, URLs без query, no bodies/secrets
Headers	Helmet, CSP, Permissions-Policy, no-store для API
CORS	same-origin SPA/API

3. Swap security gates

Перед подписью сервер обязан проверить:

- chain/token/amount allowlist;
- `walletAddress` в allowlist;
- router/executor/selectors в allowlist;
- calldata decode: tokenIn/out, recipient, amount, minOut, fee, deadline, permit/partial fill;
- баланс и allowance;
- price impact ≤ `SWAP_MAX_PRICE_IMPACT_BPS`;
- gas ≤ `SWAP_MAX_GAS_USD`;
- quote TTL/deadline;
- step-up TOTP при сумме выше threshold;
- `SWAP_EXECUTION_ENABLED` для chain;
- kill switch.

Клиент не может обойти эти проверки: сервер заново проверяет build/ready/submit.

4. Vault

- Мастер-пароль не отправляется на сервер и не хранится.
- Data key случаен; обёрнут мастер-ключом.
- Сервер хранит только salt/iterations/wrapped key/IV/ciphertext.
- При потере мастер-пароля восстановления нет.
- Поиск по vault выполняется только в браузере после unlock.
- Не логировать plaintext/ciphertext/пароль; audit пишет только id/имя/размер.

5. Заметки и файлы

- Markdown безопасный: ссылки только http/https, HTML не рендерится.
- Вложения: allowlist MIME (images/PDF/TXT/DOC/DOCX/XLS/XLSX/RAR/ZIP), 1.5 MB, 5 шт.
- Data URL проверяется по MIME/размеру; не доверять имени файла.
- Файлы хранятся в Postgres (data URL); большие объёмы увеличат БД.

6. API/SSRF

- Icon proxy разрешает только `coin-images.coingecko.com`/`assets.coingecko.com`.
- Внешние URL карточек проверяются http/https; `javascript:`/`data:` отклоняются.
- Notion import body ограничен 10 MB и content-type.
- SQL только через Drizzle/параметры; raw SQL — с параметрами.

6.1. Trading bots (paper)

- Боты не имеют доступа к приватным ключам, WalletConnect, swap build/submit и реальному ledger.
- Paper execution пишет только в `bot_*` таблицы; `transactions`, `postings`, `lots`, `trade_groups` не меняются.
- Все CRUD-мутации owner-scoped и аудируются (`bot.*`). Running-боты нельзя редактировать; стратегию с running-ботами нельзя архивировать.
- Параметры стратегий валидируются Zod; пользовательские скрипты не исполняются.
- Токены бота должны быть активными записями Базы токенов; котировки старше 5 минут игнорируются.
- Runner защищён Postgres advisory lock от двойного тика; `BOT_RUNNER_ENABLED=false` — аварийный стоп.
- Реальный live-режим требует отдельного подписанта/session key/AA и security review; правило «no private keys on server» остаётся в силе.

7. Секреты

- Не коммитить `.env`, токены, приватные ключи, seed, backup key.
- Не печатать значения в логи/скриншоты.
- При подозрении на утечку: rotate keys, revoke sessions, проверить audit/Sentry, restore при необходимости.
- `BACKUP_ENCRYPTION_KEY` хранится отдельно от дампов.

8. Ограничения

- WalletConnect/Tangem не даёт автоматической подписи; реальная автоторговля требует отдельного подписанта/AA и отдельного security review.
- Reconciliation read-only; не гарантирует полноту ledger.
- Native TRON/Solana manual-only.
- In-memory rate limit не распределён между репликами.

Troubleshooting

1. API не стартует / 502

1. Проверить Railway deployment logs.
2. `/health` и `/ready`.
3. `DATABASE_URL`, `SETUP_TOKEN`, `AUTH_ENCRYPTION_KEY`, `APP_URL` заданы.
4. Миграции применились; `nexter_app` имеет нужные гранты.
5. Нет ошибки `Chain ... is not seeded` (BSC/Ethereum обязательны; TRON/Solana optional).

2. 401/403/CSRF

- 401: сессия истекла/неверна; войти заново.
- 403 CSRF: отсутствует `x-csrf-token` или cookie `nexter_csrf`; проверить origin/APP_URL.
- 429: rate limit; подождать/проверить `auth.rate_limited` в логах.
- TOTP: проверить время на устройстве; recovery/reset.

3. Своп не проходит

Симптом	Что проверить
`SWAP_TOKEN_NOT_ALLOWED`	token в swap allowlist
`SWAP_WALLET_NOT_ALLOWED`	wallet address в `SWAP_WALLET_ALLOWLIST`
`SWAP_EXECUTION_DISABLED` / `SWAP_ETHEREUM_EXECUTION_DISABLED`	env execution flag
`SWAP_RATE_LIMITED`	Kyber 429; подождать, проверить `external_api_calls`
`SWAP_PROVIDER_UNAVAILABLE`	Kyber/RPC down
`SWAP_BUILD_FAILED` / `SWAP_SIMULATION_FAILED`	маршрут/ликвидность/газ; попробовать другую сумму/пару
`SWAP_PRICE_IMPACT_TOO_HIGH`	сумма слишком большая для пула
`SWAP_STEP_UP_REQUIRED`	ввести TOTP
`SWAP_CLOSE_AMOUNT_MISMATCH`	продажа больше остатка группы
`SWAP_CLOSE_GROUP_BUSY`	есть активный close order; дождаться/отменить
«Ожидаю ответ кошелька»	отклонить в кошельке; нажать «Отменить ожидание»
`submit_failed`	hash подписан; нажать «Повторить отправку», не подписывать заново

4. Котировки/иконки

- Нет цены: токен без CoinGecko/DexScreener или ликвидность < $25k; fallback manual price.
- Устаревшая цена: `fetched_at`; CoinGecko 10 мин, DEX 60с.
- Иконка не грузится: `logoUrl` не CoinGecko host/404; fallback initials.
- Иконка «каждый раз грузится»: проверить `Cache-Control` `/watchlist/icon`; серверный кэш 24ч.
- `market_quotes` не обновляются: `MARKET_QUOTES_ENABLED`, logs `Market quotes refreshed`, provider 429.

5. Импорт Notion

- 409 «already committed»: тот же sourceHash уже импортирован.
- `IMPORT_CONFLICT`: изменился ledger после preview/частично consumed lot; re-preview.

- Control warning: для экспорта с ID > 466 исторический baseline не применяется; проверить preview totals (ADR 0032).
- 413/415: content-type `application/x-notion-import+json`, body < 10 MB.
- Rollback 409: есть более поздние транзакции/импорт; нужен compensating reversal/cutover.

6. Задачи/заметки

- Повторяющаяся задача: content/delete — вся серия; completion — per occurrence.
- `applyToSeries=false` для статуса/переноса одного дня.
- Вложения не добавляются: MIME/размер/5 шт; data URL cap.
- Заметка не открывает вложения: list возвращает count, detail — data; проверить `GET /notes/:id`.
- Vault: забыт мастер-пароль — восстановления нет.

7. Финансы

- Неверный net worth: проверить opening balances, credit liability, отменённые операции.
- Период пустой: даты inclusive и локальная timezone.
- Импорт: preview counts/totals, rollback только последний.

7.1. Trading bots

- **Бот запущен, но сделок нет:** нет свежей котировки (`market_quotes` старше 5 минут), недостаточно истории `bot_quote_samples` (SMA/grid/DCA ждут), сработал cooldown, дневной лимит убытка, `maxPositions` или `BOT_RUNNER_ENABLED=false`.
- **All bot tokens must be active entries in the token base:** токен архивирован/удалён из Базы токенов; верните его или измените список токенов бота.
- **`Bot strategy is archived or missing / not available in this deployment`:** стратегия архивирована или её key удалён из registry. Остановите бота, создайте новую стратегию/версию.
- **`Stop running bots before archiving their strategy`:** сначала остановите всех ботов на этой стратегии.
- **`A bot with this name already exists / A strategy with this name already exists`:** имена unique per user.
- **`Budget must be a multiple of 100 USD`:** бюджет от $100 и кратен $100.
- **Изменить настройки running-бота нельзя:** остановите бота; бюджет после создания не меняется, для нового бюджета создайте нового бота или reset (reset возвращает текущий бюджет).
- **Сброс:** удаляет paper-позиции/сделки/equity snapshots, возвращает cash к бюджету и переводит бота в `stopped`. События сохраняются; отменить reset нельзя.
- **Нет equity chart:** snapshots пишутся при сделках и раз в 5 минут; подождите тик или проверьте `bot_equity_snapshots`.
- **Bot runner не тикает:** проверьте `BOT_RUNNER_ENABLED`, логи API `Bot runner tick failed`, advisory lock (`pg_locks`), что API-реплика жива и `NODE_ENV!=test`.
- **Дубли сделок:** проверьте, что несколько API-реплик используют один Postgres advisory lock; вторая реплика должна пропускать тик.

8. Reconciliation

- `RECONCILIATION_ENABLED=false` — indexer выключен.
- RPC free tier `eth_getLogs` limit — уменьшить lookback/max blocks.
- MISMATCH/MISSING — ledger vs chain; не исправлять автоматически.

9. Тесты/CI

- Локально DB-тесты skipped без `NEXTER_TEST_DB=1`/Postgres; CI — авторитет.
- E2E требует Chrome/Playwright; CI использует `PLAYWRIGHT_CHANNEL=chrome`.
- Migration drift: `db:generate`, review SQL, snapshot/journal.

- Prettier CRLF warnings на Windows не блокируют CI (committed blobs LF).

10. Диагностика

- Settings → Audit (`/settings/audit`), фильтр `swap.` / `import.` / `task.`.
- `swap_events` — state machine.
- `external_api_calls` — Kyber/CoinGecko статусы.
- Railway logs — pino JSON.
- Sentry — 5xx.
- `settings/data-quality` — negative balances/unverified/missing gas.

Гайд для AI-агентов

Read this before changing code. It is intentionally opinionated: most production incidents in this repo come from breaking one of the invariants below, not from TypeScript errors.

1. What this repo is

Private single-owner personal finance app. No public signup, no multi-tenant features. Crypto ledger + DEX swaps + personal finance (BYN) + tasks/notes/habits/vault. UI is Russian; code identifiers/comments are English.

Monorepo:

- `apps/api` — NestJS/Fastify API; also serves the built SPA.
- `apps/web` — React 19 + TanStack Router/Query + Tailwind.
- `apps/worker` — pg-boss worker.
- `apps/e2e` — Playwright acceptance tests.
- `packages/contracts` — Zod schemas/types shared by web/API.
- `packages/db` — Drizzle schema/migrations/backup scripts.
- `docs/adr`, `docs/runbooks`, `docs/project` — decisions/operations/this guide.

2. Non-negotiable rules

1. **No private keys, seed phrases, or signing keys on the server.** Ever.
2. **No public signup.**
3. **No float money.** Use `decimal.js` and Postgres `numeric`; API money values are strings.
4. **Every mutation is audited.** `AuditService` in the same transaction; critical paths use `strict: true`.
5. **Ledger is append-only.** Corrections are new transactions. Do not edit/delete posted transactions except a documented, owner-approved emergency repair with backup and audit.
6. **No secrets in code/logs/screenshots.**
7. **Schema change = ADR + migration + backup + review.**
8. **Tests/typecheck/lint must pass before merge.** Do not weaken acceptance tests to make a change pass.
9. **UI Russian, USD crypto base, BYN finance base.**
10. **BSC first, Ethereum second.**
11. **Swap tokens/wallets/routers/executors are allowlisted.**
12. **Step-up TOTP above `SWAP_STEP_UP_THRESHOLD_USD`.**
13. **No analytics/trackers.**
14. **If unclear, stop and ask.** Financial logic must not be invented.

3. Commands

```
npx pnpm@10.20.0 install
npx pnpm@10.20.0 build:packages
npx pnpm@10.20.0 dev
npx pnpm@10.20.0 format
npx pnpm@10.20.0 format:check
npx pnpm@10.20.0 lint
npx pnpm@10.20.0 typecheck
npx pnpm@10.20.0 test
npx pnpm@10.20.0 test:e2e
npx pnpm@10.20.0 build
npx pnpm@10.20.0 db:generate
npx pnpm@10.20.0 db:migrate
npx pnpm@10.20.0 backup
npx pnpm@10.20.0 restore:drill
```

Use `npx pnpm@10.20.0` on Windows if pnpm is not global. New worktree per implementation task. Do not merge with red CI.

4. Request/data flow

```
Browser → Fastify/Nest guards → controller (Zod pipe) → service → Drizzle → Postgres
                                  ↘ AuditService
                                  ↘ external providers (CoinGecko/DexScreener/Kyber/RPC)
```

- Controllers validate with Zod schemas from `@nexter/contracts`; do not duplicate types.
- Services own transactions/business rules.
- Web API calls go through `apps/web/src/lib/api.ts` and parse Zod contracts; no `any`.
- Auth: HttpOnly session cookie; mutations need `x-csrf-token`. `apiFetch` adds it.
- Rate limit and CSRF are global; import has a special auth hook/body parser.

5. Money/ledger invariants

- `transactions` + `postings` are the source of truth. Buys create `lots`; sells create `lot_consumptions` FIFO.
- `accounts` types: `wallet`, `external`, `income`, `expense`, `equity`.
- Cash equivalents (USDT/USDC) do not create lots; they are cash.
- `PricingService.latestForAssets` reads `market_quotes`, falls back to manual `prices`.
- `reporting.positions` values all wallet balances; `dashboard`/`portfolio`/`trades` share it.
- Do not calculate P&L in UI from price deltas; use API realized/unrealized fields.
- Manual adjustments only add positive lots. To reduce a non-cash position, use a sell/withdrawal or a dedicated correction design; do not fake a negative amount.
- `trade_groups` status: `open`/`partial`/`closed`/`reserve`/`for_sale`.
- Closing a group uses `closeGroupIds` + expected amount; server revalidates remaining.

6. Market data

- DexScreener primary for contract tokens with ≥ `MARKET_QUOTE_MIN_LIQUIDITY_USD` (25k), 60s.
- CoinGecko fallback/native + 7d/30d/1y/cap, 10 min.
- `fetched_at` = last check; `changed_at` = price changed.
- Icon proxy `/watchlist/icon` is exempt from `no-store`; cache 24h. Do not remove this exemption.
- `MarketQuoteRefreshService` starts 3s after boot; test env disables.
- If adding a new quote source, update `MarketQuoteService`, `PricingService`, contracts, and ADR. Do not create a separate price path.

7. Swap flow

`QUOTED → BUILT → [APPROVAL_REQUIRED] → READY → AWAITING_SIGNATURE → SUBMITTED → PENDING → CONFIRMED → ACCOUNTED`.

- Server builds/simulates Kyber calldata and verifies it before signing.
- WalletConnect/Tangem signs; server never sees keys.
- `submit_failed`: same hash must be retried; never sign twice.
- `SWAP_CLOSE_GROUP_BUSY`: do not bypass; wait/cancel/supersede only via service.
- Wallet rejection may not reject the WalletConnect promise; UI has “Отменить ожидание”. If a hash arrives after cancel, it is still submitted (never leave a broadcast tx untracked).
- `swap_events` is the diagnostic source; don't put technical order IDs in the primary UI.

8. Imports

- Notion import expects raw arrays (`source.json`, `orders.json`, `transactions.json`) and `application/x-notion-import+json`.
- Preview is non-destructive; commit writes ledger; rollback only latest committed import with no later ledger activity.
- Legacy 429-order control only applies to original export IDs ≤466. Later exports use preview totals + per-row detection (ADR 0032).
- Do not treat the control warning as an error; do not edit the hard-coded snapshot to force a new export.
- Full replacement requires clean cutover/compensating reversal, not a blind rollback.

9. Tasks/notes/vault/finance

- Recurring tasks: content fields/due shift/archive/delete apply to whole series by default (`applyToSeries`); status/completion per occurrence (`applyToSeries:false`).
- New task autosave must not create the task before recurrence is chosen (UI keeps recurrence visible until explicit save).
- Notes: Markdown is a safe subset; no HTML. List responses return attachment count, not full data URLs; detail endpoint returns full attachments.
- Vault: E2E encryption in browser; server only ciphertext. Never log/decrypt on server.
- Finance is isolated from crypto ledger; credit is liability; cancelled operations are soft-cancelled.

10. Database/migrations

- Drizzle schema: `packages/db/src/schema/index.ts`.
- Generate: `npx pnpm@10.20.0 db:generate`; review SQL and journal/snapshot.
- Migrations additive; grants for `nexter_app` may need update for new tables/columns.
- Production pre-deploy runs migrations with `MIGRATION_DATABASE_URL`; API uses `nexter_app`, worker owner role (pg-boss DDL).
- `nexter_app` has SELECT/INSERT/UPDATE and limited DELETE; do not add DELETE grants casually.
- Manual data seeds need journal entries and ADR.

11. Frontend patterns

- Routes: `apps/web/src/app/router.tsx` (TanStack Router). App layout in `root-layout.tsx`.
- Data: `useQuery` / `useInfiniteQuery` with query keys; `apiFetch` / `apiPost` typed by contracts.
- Drawers/dialogs: shared components; reference-counted scroll lock. Dialogs close on backdrop/X/Escape; do not reintroduce footer "Готово" unless asked.
- `LiveValue` flashes only changed values (not whole cards). `PriceFreshness` shows fetched/changed/source.
- Token icons: `TokenIcon` + `tokenIconSrc`; icon URL is `/api/v1/watchlist/icon?url=...`.
- Russian copy; use existing `format*` helpers; never `Number()` for money except display-only.
- New UI should be full-width rows/cards where the owner asked (watchlist, vault, notes, tasks).

12. Testing

- Unit: `*.spec.ts` next to source. API DB tests are skipped without `NEXTER_TEST_DB=1`/Postgres; CI runs them.
- E2E: Playwright, Chrome channel in CI (`PLAYWRIGHT_CHANNEL=chrome`). Do not change acceptance tests to hide a regression; update only when product behavior intentionally changes.
- Parser tests: `notion-import.parser.spec.ts`.
- Web tests: `apps/web` Vitest.
- CI runs migration twice, static/API smoke, Docker API/worker, backup restore drill, encrypted backup smoke, dependency audit. A red E2E step skips later checks — fix it, don't merge.

13. Production/deploy

- Railway API URL: `https://nexterapi-production.up.railway.app`.
- API serves SPA; frontend-only changes require API service redeploy (watch patterns include `apps/web/**`, `packages/**`).
- Before production migration/data repair: encrypted backup + restore check; record IDs/paths.
- Use `railway ssh` into Postgres for dumps/checks; do not print DB passwords.
- Production secrets in Railway; backup key outside repo (`~/.nexter/backup-key` local).
- Worker remains owner role because pg-boss DDL.
- `ENABLE_DOCS` only dev/test.

14. Common pitfalls

- **Drizzle nullable columns**: `eq(table.col, maybeNull)` type errors; cast/narrow explicitly.
- **Stale closures**: `phase` in async execute/catch can be stale; use refs for state read after await.
- **Wallet promises can hang**: always offer a cancel/recovery path; don't rely only on provider rejection.
- **Date-only vs timestamp**: tasks due dates use local noon; cursor timestamps are ms; don't mix.
- **CRLF on Windows**: Prettier warnings are okay; committed blobs are LF.
- **Contracts package dist**: after editing `packages/contracts`, build it before API/web typecheck if types don't update.
- **API role**: new tables need grants; worker/API roles differ.
- **No `any`** in web API layer; contracts should be Zod-first.
- **XSS/SSRF**: validate URLs/Markdown/icon hosts; do not fetch arbitrary URLs.
- **Attachment size**: base64 expands 4/3; 1.5 MB file can exceed 2 MB data URL; current cap 2.1 MB.
- **`market_quotes` fallback**: don't overwrite CoinGecko periods with null on DEX ticks; preserve unless provider explicitly returns null on a due tick.
- **Swap preview**: Do not poll Kyber too aggressively; 500ms debounce/15s poll/5s server cache.
- **Order footer**: technical IDs belong in audit/logs, not the primary UI.

15. Trading bots (implemented, see `trading-bots.md`)

- Paper-only v1; no real swaps/signing. Tables are `bot_strategies`, `trading_bots`, `bot_tokens`, `bot_positions`, `bot_trades`, `bot_equity_snapshots`, `bot_events`, `bot_quote_samples` (migration `0032`).
- Strategy algorithms live in `apps/api/src/modules/trading-bots/strategies/` as a code registry with Zod params. New strategy = code + unit tests + deploy; never execute user scripts.
- `BotRunnerService` ticks every `BOT_RUNNER_INTERVAL_SECONDS` (60) under Postgres advisory lock `nexter:bot-runner`; `runOnce()` is used by tests. Quotes come only from `market_quotes`; stale (>5 min) or missing quotes are skipped.
- Sizing: `min(budget * positionSizePct / 100, cash)` per buy; no per-asset cap, so DCA/grid can add until strategy `maxAdds` or cash runs out. Max positions applies only to new positions.
- P&L is average-cost: buy cost basis includes fee; sell realized = net proceeds − cost basis. Never write paper trades into `transactions`/`postings`/`lots`/`trade_groups`.
- All user mutations are audited (`bot.*`); paper trades/events are their own journals. Strategy archive is blocked while a bot is running; running bots cannot be edited and archived strategies cannot be started.
- API under `/api/v1/trading-bots`; UI at `/testing` with full-width bot cards, fan, drawer stats, strategies tab.
- Real automated trading is blocked by WalletConnect/Tangem manual signing. A future live venue needs an owner-approved signer/session-key/AA design and a separate security review; no server private keys.

16. Definition of Done

- Spec/acceptance clear; tests added/updated.
- `format:check`, `lint`, `typecheck`, `test`, `build` pass.
- Migration reviewed, backup/restore plan if schema/data.

- Audit written for mutations.
- Docs/ADR/runbook updated.
- No secrets.
- Security review for swap/vault/import/bot changes.
- Feature flag/rollback plan where relevant.

Торговые боты (design)

> Статус: **реализовано 2026-11-12**. Миграция `0032` (аддитивная) добавляет изолированные таблицы ботов. Production-миграция и деплой выполняются отдельным шагом после проверки.
>
> Это paper-only система. Реальные свопы, подписи и приватные ключи не используются. Бумажные сделки не пишут в `transactions` / `postings` и не влияют на реальный ledger, отчёты и налоги.

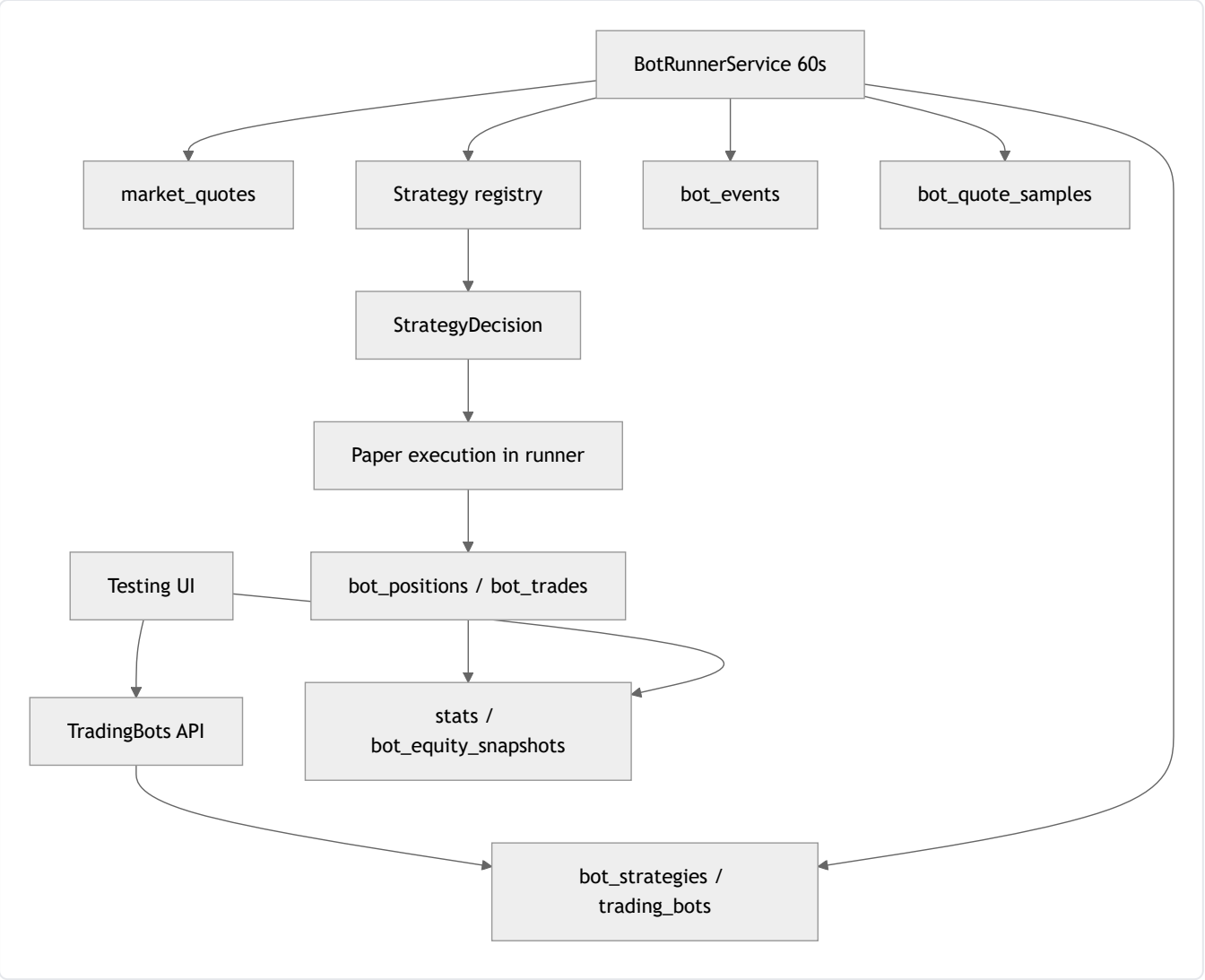
1. Что умеет система

- Несколько независимых ботов; у каждого своя стратегия, бюджет, список токенов и риск-параметры.
- Боты работают на сервере по минутному тику, даже когда браузер закрыт.
- Виртуальные покупки/продажи по последним `market_quotes` с симулированными комиссией и проскальзыванием.
- Открытые позиции, история сделок, журнал событий, equity curve.
- Статистика: realized/unrealized/total P&L, cash/equity, profit factor, win rate, wins/losses, число сделок, max drawdown.
- Запуск/остановка, сброс бумажного состояния, архивирование.
- Стратегии — встроенный реестр алгоритмов; владелец создаёт экземпляр с названием, описанием и параметрами.

2. Границы v1

- **Paper only.** Нет реальных транзакций и подписей.
- **Long-only spot.** Без шортов, плеча, маржи и деривативов.
- **Minute tick.** Решение раз в 60 секунд по `market_quotes`; цена исполнения — последняя котировка с симулированным slippage.
- **No backtest.** История начинается с первого запуска; исторический бэктест — отдельный будущий этап.
- **No arbitrary scripts.** Алгоритмы только в коде; параметры валидируются Zod-схемой стратегии.
- **Isolated tables.** `trading_bots`, `bot_*` не связаны с реальным ledger, кроме ссылок на `assets` и `users`.
- **Single tick source.** Цены берутся только из `market_quotes`; отдельного источника цен у ботов нет.

3. Компоненты



- `TradingBotsService` — CRUD, статистика, сброс/архив, аудит.
- `BotRunnerService` — интервальный сервис API; advisory lock, минутный тик, paper execution, snapshots, retention.
- `strategies/` — реестр из пяти алгоритмов и общие индикаторы.
- `market_quotes` — единственный источник цены и 24h change.
- `bot_quote_samples` — минутные точки для индикаторов; общие для всех ботов.

4. Модель данных

Все таблицы owner-scoped через `trading_bots.user_id`; дочерние таблицы каскадно удаляются вместе с ботом. `bot_strategies` удаляются только soft archive (`archived_at`).

`bot_strategies`

Поле	Тип	Смысл
`id`	uuid	PK
`user_id`	uuid	owner
`key`	text	ключ алгоритма из registry
`name`	text	имя экземпляра, unique среди неархивных
`description`	text	описание
`version`	int	версия алгоритма
`params`	jsonb	валидированные параметры
`archived_at`	timestamptz	soft archive
`created_at/updated_at`	timestamptz	

`key` обязан существовать в registry на момент создания и запуска. Параметры проверяются Zod-схемой; неизвестный ключ или невалидные параметры дают `VALIDATION_ERROR`.

`trading_bots`

Поле	Тип	Смысл
`id`	uuid	PK
`user_id`	uuid	owner
`strategy_id`	uuid	→ `bot_strategies`
`name`	text	unique среди неархивных
`description`	text	
`status`	text	`draft`, `running`, `stopped`
`budget_usd`	numeric(24,8)	стартовый бюджет, неизменяемый после создания
`cash_usd`	numeric(24,8)	свободный бумажный кэш
`max_positions`	int	максимум одновременно открытых позиций
`position_size_pct`	numeric(5,2)	% бюджета на одну покупку
`fee_bps`	int	симулированная комиссия (25 = 25 bps)
`slippage_bps`	int	симулированное проскальзывание (10 = 10 bps)
`max_daily_loss_pct`	numeric(5,2)	дневной лимит реализованного убытка
`cooldown_minutes`	int	пауза между покупками по токену
`peak_equity_usd`	numeric(24,8)	пик equity для max drawdown
`max_drawdown_pct`	numeric(5,2)	максимальная просадка, обновляется runner
`started_at/stopped_at/last_tick_at`	timestamptz	
`archived_at`	timestamptz	soft delete
`created_at/updated_at`	timestamptz	

Бюджет — от $100 и кратен $100 (шаг $100). `position_size_pct` — доля **бюджета** на одну покупку, а не доля свободного кэша; DCA/grid могут делать несколько покупок, пока хватает кэша и не достигнут `maxAdds` стратегии. Один экземпляр стратегии может использоваться несколькими ботами; изменение её параметров влияет на всех этих ботов.

`bot_tokens`

`bot_id`, `asset_id`, `enabled`, `created_at`. `(bot_id, asset_id)` unique. Токены выбираются только из активной Базы токенов (`watchlist`); дубликаты отклоняются. Один токен может быть у нескольких ботов; paper-портфели независимы.

`bot_positions`

`bot_id`, `asset_id`, `amount numeric(36,18)`, `avg_cost_usd`, `cost_basis_usd`, `opened_at`, `updated_at`. `(bot_id, asset_id)` unique. Учёт по средней цене (average cost), не FIFO. `cost_basis_usd` включает комиссию покупки.

`bot_trades`

`bot_id`, `asset_id`, `side` (`buy`/`sell`), `amount`, `price_usd`, `usd_value`, `fee_usd`, `slippage_usd`, `realized_pnl_usd`, `reason`, `signal jsonb`, `executed_at`. `price_usd` — фактическая fill-цена с slippage. Для sell `realized_pnl_usd = net proceeds − cost_basis_usd`.

`bot_equity_snapshots`

`bot_id`, `captured_at`, `cash_usd`, `positions_value_usd`, `equity_usd`, `realized_pnl_usd`, `unrealized_pnl_usd`. Снимок при каждой сделке и не реже одного раза в 5 минут. Retention — 1 год.

`bot_events`

`bot_id`, `type` (`created`, `started`, `stopped`, `buy`, `sell`, `error`, `reset`, `archived`), `message`, `payload jsonb`, `created_at`. Retention — 90 дней.

`bot_quote_samples`

`asset_id`, `price_usd`, `change_24h_pct`, `captured_at`. Одна точка на токен за тик (не чаще 55 секунд), общая для всех ботов. Retention — `BOT_SAMPLE_RETENTION_DAYS` (30 дней по умолчанию). Индексы: `(asset_id, captured_at)` и `(captured_at)` для очистки.

5. Стратегии

5.1. Интерфейс

```
interface StrategyContext {
  now: Date;
  params: Record<string, unknown>;
  priceUsd: string;
  change24hPct: string | null;
  history: Array<{ priceUsd: string; capturedAt: Date }>;
  position: { amount: string; avgCostUsd: string; costBasisUsd: string } | null;
  buyCount: number; // покупок после последней продажи по этому токену
}

interface StrategyDecision {
  action: 'buy' | 'sell' | 'hold';
  reason: string;
}

interface StrategyDefinition {
  key: string;
  version: number;
  name: string;
  description: string;
  defaultParams: Record<string, unknown>;
  parseParams(value: Record<string, unknown>): Record<string, unknown>;
  evaluate(ctx: StrategyContext): StrategyDecision;
}
```

Новая стратегия: файл в `apps/api/src/modules/trading-bots/strategies/`, Zod-схема параметров, регистрация в `strategyDefinitions`, unit-тесты и deploy. UI автоматически показывает новый шаблон.

5.2. Встроенные стратегии

key	Название	Логика входа	Логика выхода	Параметры по умолчанию
`momentum_breakout`	Импульсный пробой	fast SMA > slow SMA и 24h change ≥ порога	тейк-профит, стоп-лосс, fast SMA ниже slow SMA	fast 5, slow 20, minChange24h 1%, TP 5%, SL 3%
`mean_reversion`	Возврат к среднему	цена ниже SMA на `entryDropPct`	возврат к SMA, TP, SL	lookback 20, entryDrop 3%, exitAbove 0.5%, TP 5%, SL 5%
`grid_range`	Сетка в диапазоне	цена ниже rolling high на шаг сетки	шаг вверх от средней, SL; докупка на следующей ступени	anchor 20, gridStep 1.5%, maxAdds 4, TP 1.5%, SL 6%
`trend_pullback`	Откат в тренде	fast > slow и откат к fast SMA	TP, SL, пробой тренда	fast 8, slow 30, pullback 1%, TP 5%, SL 3%
`dca_dip`	Докупка на просадке	цена ниже rolling high/ средней на `dipPct`	TP, SL; докупка до `maxAdds`	anchor 20, dip 3%, maxAdds 3, TP 5%, SL 8%

Все стратегии long-only и работают по одному токену независимо. Общий risk layer в runner: max positions, position size, cash, cooldown, daily loss limit, freshness quote (≤5 минут).

5.3. Как добавить новую стратегию

1. Создать `apps/api/src/modules/trading-bots/strategies/<key>.ts`.
2. Описать Zod-схему параметров с безопасными default-значениями и `evaluate(ctx): StrategyDecision`.
3. Добавить стратегию в `strategyDefinitions` в `strategies/index.ts`.
4. Добавить unit-тесты на buy/sell/hold и граничные параметры.

5. Прогнать `format`, `typecheck`, `lint`, `test`, `build`.
6. Задеплоить API. UI автоматически покажет новый шаблон в разделе `Стратегии`.
7. Создать экземпляр стратегии и бота. Существующие боты не меняются.

`key` — стабильный контракт. `version` фиксируется при создании экземпляра, но runner исполняет текущий код registry; обратно несовместимое изменение поведения лучше выпускать новым `key` или отдельной миграцией экземпляров. Любое изменение смысла параметров описывается в ADR/`docs/decisions.md`.

5.4. Paper execution

- **Buy:** `usd = min(budget * positionSizePct / 100, cash)`; `fillPrice = quote * (1 + slippageBps/10000)`; `amount = usd / fillPrice`; `fee = usd * feeBps/10000`; `debit = usd + fee`; cost basis увеличивается на `debit`. Позиция открывается только если `openPositions < maxPositions`; докупка существующей позиции не увеличивает счётчик позиций.
- **Sell:** продаётся вся позиция; `fillPrice = quote * (1 - slippageBps/10000)`; `gross = amount * fillPrice`; `fee = gross * feeBps/10000`; `net = gross - fee`; `realized = net - costBasis`; кэш увеличивается на `net`.
- **Cooldown:** блокирует только покупки по токену в течение `cooldownMinutes` после последней покупки; продажи не блокируются.
- **Daily loss:** если реализованный убыток за локальный день ≥ `budget * maxDailyLossPct / 100`, новые покупки блокируются до следующих суток; продажи/стопы продолжают работать.
- **Quote freshness:** котировка старше 5 минут или цена ≤ 0 — токен пропускается на этом тике.
- **No quote:** токен пропускается; событие `error` не создаётся, чтобы не засорять журнал.
- **Fee/slippage** записываются в сделку; `slippage_usd` — разница между ценой без slippage и фактической fill-ценой.

5.5. Статистика

- `realizedPnlUsd` — сумма `realized_pnl_usd` по sell-сделкам.
- `unrealizedPnlUsd` — сумма `(currentPrice * amount − costBasis)` по открытым позициям; без котировки позиция даёт `null`, в сумму входит 0.
- `equityUsd = cashUsd + positionsValueUsd`.
- `totalPnlUsd = equityUsd − budgetUsd`.
- `grossProfitUsd`/`grossLossUsd` — только по sell-сделкам.
- `profitFactor = grossProfit / grossLoss`; `null`, если убытков не было.
- `winRate = wins / (wins + losses) * 100`; сделки с нулевым P&L не считаются ни победой, ни убытком.
- `maxDrawdownPct` — максимальная просадка; runner обновляет `peak_equity_usd`/`max_drawdown_pct` при каждом snapshot, поэтому статистика не зависит от лимита выборки.
- `tradesCount` включает buy и sell.

6. Runner и надёжность

- `BotRunnerService` стартует через 5 секунд после boot API и затем каждые `BOT_RUNNER_INTERVAL_SECONDS` (60). Для индикаторов загружаются последние 300 минутных samples (покрывает максимальные окна стратегий).
- `BOT_RUNNER_ENABLED=true` по умолчанию; `NODE_ENV=test` отключает интервал.
- Тик защищён session-level Postgres advisory lock `nexter:bot-runner` на отдельном соединении из пула; вторая реплика/второй процесс пропускает тик. Unlock выполняется в `finally`; при ошибке соединение возвращается в пул после попытки unlock.
- Внутрипроцессный guard `running` предотвращает наложение тиков.
- Ошибка одного бота не останавливает остальных; пишется `bot_events.error`.
- `runOnce()` — публичный метод для тестов/ручного запуска; интервал вызывает его же.
- Retention: samples — `BOT_SAMPLE_RETENTION_DAYS` (30), events — 90 дней, equity — 365 дней. Очистка выполняется на каждом тике.
- При нескольких API-репликах advisory lock предотвращает дубли; per-bot lock не нужен, пока тик один.

7. API

Базовый путь: `/api/v1/trading-bots`. Все мутации требуют CSRF и аудируются.

Метод	Путь	Описание
GET	`/strategies`	шаблоны registry + созданные экземпляры
POST	`/strategies`	создать экземпляр (`key`, `name`, `description`, `params`)
PATCH	`/strategies/:id`	изменить имя/описание/параметры/архив
GET	`/bots?includeArchived=`	список ботов + статистика
POST	`/bots`	создать бота
GET	`/bots/:id`	детали: бот, стратегия, токены, позиции
PATCH	`/bots/:id`	настройки; только не-running
POST	`/bots/:id/start`	запуск
POST	`/bots/:id/stop`	остановка (позиции сохраняются)
POST	`/bots/:id/reset`	сброс paper-состояния к бюджету
DELETE	`/bots/:id`	soft archive
GET	`/bots/:id/positions`	не отдельный endpoint; позиции в `/bots/:id`
GET	`/bots/:id/trades?limit=`	сделки
GET	`/bots/:id/events?limit=`	журнал
GET	`/bots/:id/equity?limit=`	equity curve

Ошибки: `VALIDATION_ERROR`, `CONFLICT`, `NOT_FOUND`. Уникальность имени бота/стратегии — `CONFLICT`. Running-бот нельзя редактировать; стратегию с running-ботами нельзя архивировать. Архивированную/недоступную стратегию нельзя запустить.

Аудит: `bot.strategy_created`, `bot.strategy_updated`, `bot.created`, `bot.updated`, `bot.started`, `bot.stopped`, `bot.reset`, `bot.archived`. Бумажные сделки и события живут в `bot_trades` / `bot_events`, а не в `audit_log`.

8. UI

- Top-level пункт **Тестирование** (`/testing`).
- Табы **Роботы** и **Стратегии**.
- Роботы — full-width карточки: имя, статус, стратегия, описание, бюджет, equity, P&L, profit factor, win rate, число сделок, открытые позиции; fan крутится у running-ботов.
- Создание/редактирование: имя, описание, стратегия, токены из Базы токенов, бюджет $1000 с шагом $100, max positions, position size %, fee/slippage bps, daily loss %, cooldown.
- Drawer бота: equity chart, открытые позиции с текущей ценой/P&L, сделки, журнал событий, start/stop/reset/archive.
- Стратегии: шаблоны registry и созданные экземпляры; создание из шаблона, редактирование параметров JSON, архив.
- Fan в шапке раздела и на карточках; live-refresh индикатор учитывает ключи `trading-bot*`.

9. Безопасность и границы

- Нет приватных ключей, seed, подписи, WalletConnect и реальных свопов.
- Paper-сделки не создают `transactions`, `postings`, `lots`, `trade_groups`; реальный ledger и налоги не меняются.
- Все запросы owner-scoped; чужие боты/стратегии недоступны.

- Токены бота должны быть активными записями Базы токенов.
- Стратегии — только код; никакого исполнения пользовательских скриптов.
- Реальная автоторговля невозможна через WalletConnect/Tangem без ручного подтверждения каждой подписи. Для live нужен отдельный подписант/session key/AA, отдельный security review и ADR; сервер не должен хранить приватные ключи.

10. Ограничения и будущее

- Минутный тик — не HFT; стоп-лосс срабатывает с задержкой до 60 секунд.
- Paper fill не учитывает глубину пула; slippage фиксированный.
- Токены без котировки/свежести пропускаются.
- Нет исторического бэктеста; equity curve начинается с запуска.
- Нет уведомлений вне приложения; журнал событий внутри drawer.
- Будущий live-режим: отдельный `ExecutionVenue` adapter, allowlist, симуляция, лимиты, kill switch, аудит и подписант на стороне владельца. До этого боты остаются paper.

11. Проверка

- Unit: `apps/api/src/modules/trading-bots/strategies/strategies.spec.ts` — реестр, параметры, buy/sell/hold для пяти стратегий.
- E2E: `apps/api/test/trading-bots.e2e.spec.ts` — создание стратегии/бота, валидация бюджета, buy/sell по котировкам, статистика, reset, archive.
- Перед production: format, typecheck, lint, test, build; backup + restore check; миграция `0032`; проверка `/ready` и грантов `nexter_app`.